

Robotics Manipulation & Control

Kinematic and Dynamic Analysis of YARO 1105 robot (6-DOF)

Summary. This report covers the full kinematics-to-dynamics pipeline for a 6-DOF manipulator with a spherical wrist: MDH parameter derivation, forward and closed-form inverse kinematics, the geometric Jacobian, a quintic joint-space trajectory through five Cartesian waypoints, and joint torque profiles with and without a 5 kg payload. Every stage is independently verified: FK/IK round-trip error below 10^{-6} , the hand-derived Jacobian matching `jacob0()` to 10^{-8} , and a from-scratch RNEA implementation cross-checked against energy and power conservation to 10^{-8} – 10^{-9} . That RNEA implementation was necessary rather than optional: applying `roboticstoolbox`'s built-in dynamics directly to the MDH model gives a home-pose shoulder torque of 7.09 N·m where the correct value is 0.0065 N·m, a frame-misalignment error most workflows would miss silently.

Robot Modeling

Robot Structure and MDH Parameters

Assumptions Made

- **Base Frame & Home Position:** Z_0 is aligned with the Joint 1 rotation axis (pointing upward), with X_0 pointing arbitrarily forward; the zero configuration ($q_i = 0$) has the arm pointing vertically upward.
- **Twist Angles (α):** determined by the physical, orthogonal layout of the motors, independent of frame choice, requiring 90° or 180° twists to orient each Z-axis to the next hinge pin.
- **θ_i Offsets:** arise separately, purely from the X-axis convention ($X_i = Z_i \times Z_{i+1}$, right-hand rule) used to derive X_1 through X_5 ; these offsets cancel out of the final motor commands (see Inverse Kinematics).
- **End Effector Frame:** Since there is no Z_7 axis, X_6 is chosen to align with X_5 , giving a 0° offset for the final frame.

Verified MDH Parameter Table

i	α_{i-1}	a_{i-1} (m)	d_i (m)	θ_i offset
1	0°	0	0.3035	180°
2	90°	0	0.082	90°
3	180°	0.49	0.082	90°
4	90°	0	0.4703	0°
5	90°	0	0	180°
6	90°	0	0	0°

Table 1: Modified Denavit-Hartenberg (MDH) Parameters

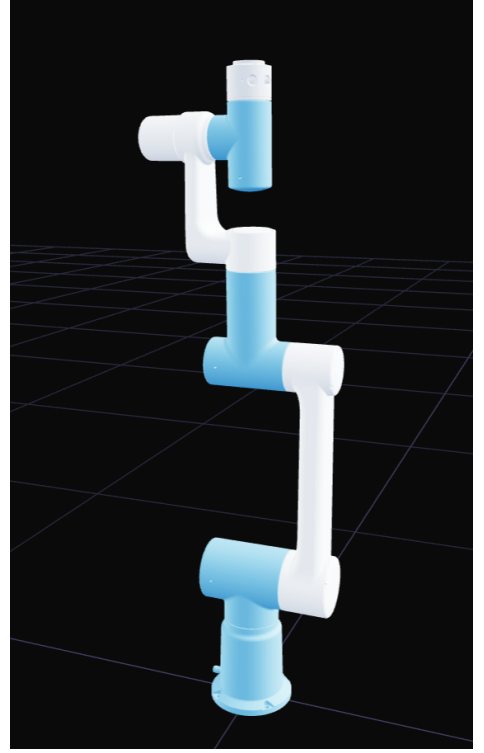


Figure 1: Overall Robot Model

Brief Explanation of Derivation

- **Link Lengths (a):** The only non-zero a parameter occurs at a_3 (0.49 m). This represents the physical length of the upper arm, as Z_2 and Z_3 (the shoulder and elbow) are perfectly parallel and do not intersect.
- **Link Offsets (d):** The d parameters represent physical translations along the Z-axes. For example, d_4 (0.4703 m) is the combined physical length of the forearm, calculated by adding the URDF lengths after the frame twist aligns them into a single linear vector.
- **The Spherical Wrist:** Joints 4, 5, and 6 form a spherical wrist. Because their Z-axes physically intersect at a single mathematical point in space (the wrist center), the required translation along Z_5 to reach the next frame is zero.
- **Twist Angles (α):** The α parameters represent the orthogonal layout of the physical motors, requiring 90° or 180° twists to orient the current Z-axis to the subsequent hinge pin.

Note: A fixed tool transform (frame 6 $\rightarrow$ end-effector) is applied as a pure translation (0, 0.000532, 0.1447) with an identity rotation. This is standard practice, frame 6 sits at the joint-6 geometry, and one more fixed offset reaches the actual flange/tool point.

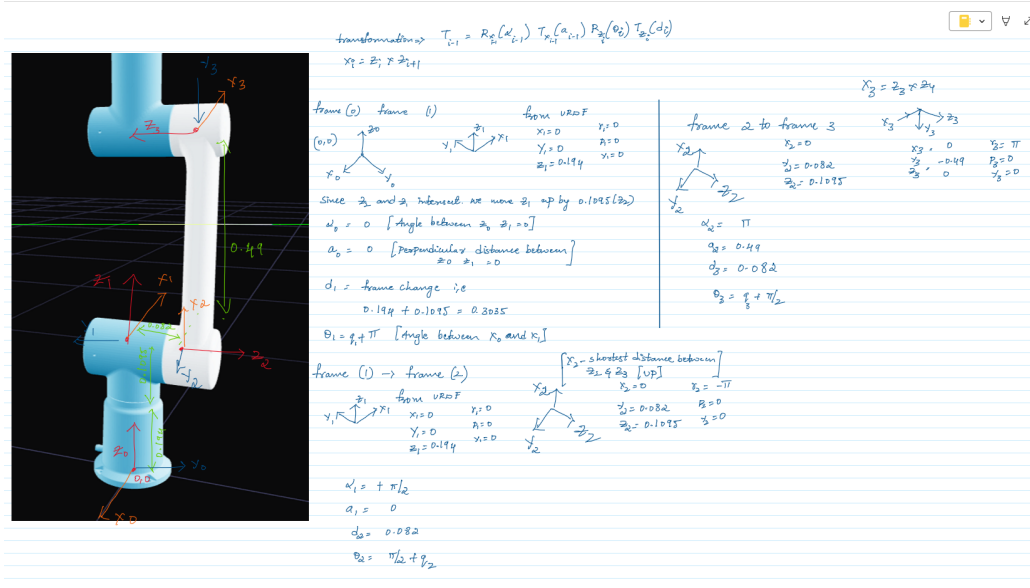


Figure 2: Handwritten derivation steps for base and lower arm frames.

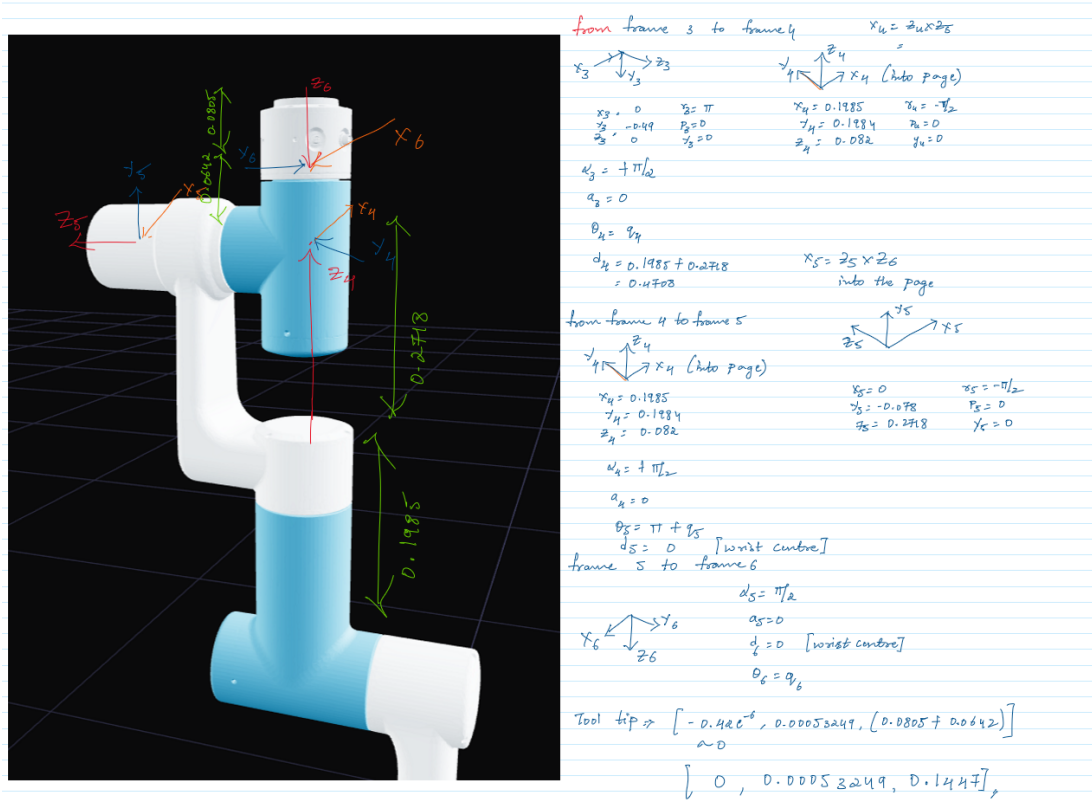


Figure 3: Handwritten derivation steps for the spherical wrist and end-effector.

Forward Kinematics

Notation. We write T_i^{i-1} for the homogeneous transformation that expresses frame i in the coordinates of frame $(i-1)$ i.e., the transform from frame $(i-1)$ to frame i consistent with Craig's Modified DH (MDH) convention:

$$T_i^{i-1} = R_x(\alpha_{i-1}) \cdot T_x(a_{i-1}) \cdot R_z(\theta_i) \cdot T_z(d_i)$$

Construction of Forward Kinematics

Using the derived MDH parameters, forward kinematics chains each per-link transform T_i^{i-1} from the fixed base to the end-effector. The kinematic chain up through the last actuated joint (frame 6) is the sequential product of these six matrices:

$$T_6^0 = T_1^0 \cdot T_2^1 \cdot T_3^2 \cdot T_4^3 \cdot T_5^4 \cdot T_6^5$$

Because joints 4, 5, and 6 form a spherical wrist (their axes intersect at a common point, with $a_3 = a_4 = a_5 = 0$), frame 6's origin coincides with this wrist center rather than with the physical tool tip. The actual end-effector pose therefore requires one additional fixed transformation, T_{tool} , representing the constant offset from frame 6 to the tool point:

$$T_{ee}^0 = T_6^0 \cdot T_{tool}$$

T_{tool} is a pure translation (no rotation) and was determined by solving $T_{tool} = (T_6^0)^{-1} \cdot T_{ee}^0$ at the zero configuration, using the end-effector pose obtained by chaining the URDF's own joint origins as ground truth. This two-stage structure, a configuration-dependent joint chain followed by a fixed tool offset; is standard practice, since it separates the robot's actuated geometry from the fixed hardware attached at its wrist.

Purpose of Homogeneous Transformation Matrices

Homogeneous matrices are used because they package two distinct mathematical operations, 3D rotation (a 3×3 matrix) and 3D translation (a 3×1 vector); into a single 4×4 algebraic structure. This lets a multi-joint spatial displacement be computed via ordinary matrix multiplication (as chained above), rather than through separate, non-uniform rotation and translation steps at each link.

Information in the Final Transformation Matrix

The final matrix, T_{ee}^0 , completely defines the end-effector's pose relative to the base frame for a given set of joint angles. It contains two distinct pieces of information:

- **Position:** The top-right 3×1 vector gives the Cartesian coordinates (x, y, z) of the tool point in the base frame.
- **Orientation:** The top-left 3×3 submatrix gives the rotation matrix describing the end-effector's orientation, from which Euler angles or a quaternion can be extracted if a different orientation representation is needed.

Analytical Inverse Kinematics

Using the MDH parameters derived earlier: since the robot has a spherical wrist, a closed-form solution is obtainable. The challenge of Inverse Kinematics (IK) is fundamentally one of reverse engineering: given a desired position and orientation for the end-effector in 3D space, we solve for precise joint angles required to achieve it.

Because this robot features a spherical wrist (the axes of joints 4, 5, and 6 intersect at a single point), we can employ kinematic decoupling. This slices the complex 6-DOF problem into two independent, solvable 3-DOF problems:

- **Inverse Position:** Using the first three joints to place the wrist center exactly at the target coordinates.
- **Inverse Orientation:** Using the final three joints to rotate the tool into the correct posture, anchored at that wrist center.

Part 1: Position IK (Joints 1, 2, 3)

The position problem can be visualized by collapsing the 3D workspace into a simplified 2D plane. Before the arm can reach for the target, it must first aim the base (q_1) toward the target's (x, y) coordinates. Once the base is aligned, the problem becomes strictly two-dimensional: the target's location in this plane forms a triangle defined by the robot's physical linkages and the computed distance to the target.

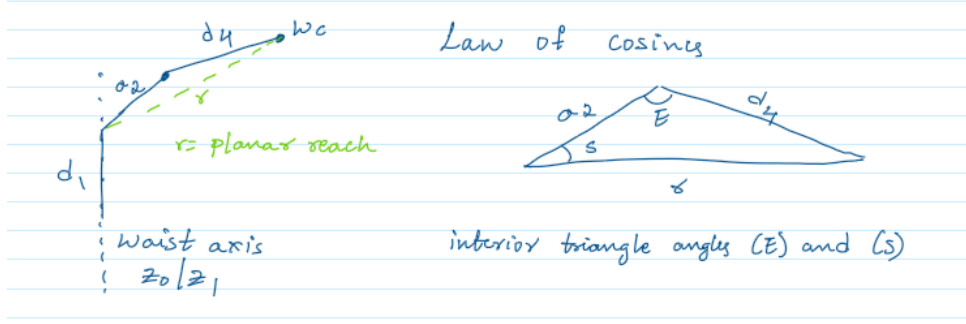


Figure 4: 2D planar projection of the manipulator (top) and the geometric triangle formed by the upper arm, forearm, and reach (bottom), used to solve q_2 and q_3 . Note: the two labeled triangle angles are the physical interior angles $\angle E$ (elbow vertex) and $\angle S$ (shoulder vertex); not the joint values q_2, q_3 directly; the offset between them is handled in the equations below.

Crucial Geometry Note: The lateral shoulder offsets ($d_2 = 0.082$, $d_3 = 0.082$) perfectly cancel due to a 180° twist ($\alpha_2 = 180^\circ$) in the MDH table. The algebraic derivations below naturally absorb the MDH θ offsets (including q_1 's 180° offset and the 90° offsets on joints 2 and 3); the equations output the pure, final motor commands directly, with no offset correction needed afterward.

Given a target wrist-center position (x_c, y_c, z_c) :

1. Base angle

$$q_1 = \text{atan2}(y_c, x_c)$$

2. Planar reach geometry

- Horizontal reach: $r = \sqrt{x_c^2 + y_c^2}$
- Vertical height: $Z = z_c - d_1$
- Distance from shoulder to wrist: $L = \sqrt{r^2 + Z^2}$
- Angle of reach from vertical Z-axis: $\theta_{\text{dashed}} = \text{atan2}(r, Z)$

3. Internal triangle angles (Law of Cosines)

$$\begin{aligned}\angle E &= \arccos\left(\frac{a_2^2 + d_4^2 - L^2}{2a_2d_4}\right) \\ \angle S &= \arccos\left(\frac{a_2^2 + L^2 - d_4^2}{2a_2L}\right)\end{aligned}$$

4. Final shoulder and elbow angles

Branch 1:

$$q_3 = +(\pi - \angle E), \quad q_2 = \theta_{\text{dashed}} + \angle S$$

Branch 2:

$$q_3 = -(\pi - \angle E), \quad q_2 = \theta_{\text{dashed}} - \angle S$$

(Reachability: if the law-of-cosines argument exceeds ± 1 , the target is outside the arm's workspace.)

Part 2: Orientation IK (Joints 4, 5, 6)

With the wrist center fixed by the first three joints, the remaining task is bridging the rotational gap between the arm's current posture and the target tool orientation. By analyzing specific entries of the required rotation matrix, we can extract the wrist's Euler-like angles.

Given a target tool orientation R_{target} :

1. Required wrist rotation

Compute the rotation matrix for the first three joints (R_0^3), then extract the wrist's required rotation:

$$R_3^6 = (R_0^3)^T \cdot R_{\text{target}}$$

2. Extract angles from R_3^6 (0-indexed matrix)

Once R_3^6 is computed, multiplying out the physical twists of the wrist (the 90° MDH rotations) gives:

$$R_3^6 = \begin{bmatrix} s_4 s_6 - c_4 c_5 c_6 & s_4 c_6 + s_6 c_4 c_5 & -c_4 s_5 \\ s_5 c_6 & -s_5 s_6 & -c_5 \\ -s_4 c_5 c_6 - s_6 c_4 & s_4 s_6 c_5 - c_4 c_6 & -s_4 s_5 \end{bmatrix}$$

Because the middle row and right column of R_3^6 each involve only two of the three wrist angles, the system decouples: q_5 is recovered first (via $R[1, 2] = -\cos q_5$, with sign ambiguity resolved through $\sin q_5 = \pm \sqrt{R[1, 0]^2 + R[1, 1]^2}$), then q_6 and q_4 follow by dividing the remaining middle-row and right-column entries by $\sin q_5$.

Sign consistency: if the negative branch was chosen for $\sin q_5$ above, the signs of the inputs to both the q_6 and q_4 formulas below must also flip, dividing by a negative number reverses both components.

3. Combine into atan2 form

$$q_5 = \text{atan2}\left(\pm \sqrt{R[1, 0]^2 + R[1, 1]^2}, -R[1, 2]\right)$$

$$q_6 = \text{atan2}(-R[1, 1], R[1, 0]), \quad q_4 = \text{atan2}(-R[2, 2], -R[0, 2]) \quad (\text{no-flip branch})$$

$$q_6 = \text{atan2}(R[1, 1], -R[1, 0]), \quad q_4 = \text{atan2}(R[2, 2], R[0, 2]) \quad (\text{flip branch})$$

4. Singularity fallback (gimbal lock)

If $s_5 \approx 0$ ($q_5 \approx 0^\circ$ or 180°): axes Z_3 and Z_6 align, and only $q_4 \pm q_6$ is determined.

- Lock q_4 to its previous known value (or 0).
- $q_6 = \text{atan2}(R[0, 1], R[0, 0]) - q_4$

Result: Up to 4 total solutions (2 position branches $\times$ 2 orientation branches). Select by joint-limit feasibility or proximity to the current pose.

The Robot Jacobian and Velocity Mapping

The **Jacobian matrix** $J(q)$ is a configuration-dependent matrix that bridges the gap between joint space and Cartesian space. While Forward Kinematics determines the *position* of the end-effector based on joint angles, the Jacobian determines the *velocity* of the end-effector based on joint velocities.

Mathematically, it acts as a linear transformation mapping the joint velocity vector $\dot{q}$ to the end-effector's spatial velocity V (comprising linear velocity v and angular velocity ω):

$$V = \begin{bmatrix} v \\ \omega \end{bmatrix} = J(q) \dot{q}$$

Where:

- $\dot{q} = [\dot{\theta}_1, \dot{\theta}_2, \dot{\theta}_3, \dot{\theta}_4, \dot{\theta}_5, \dot{\theta}_6]^T$ are the velocities of the six motors.
- $v = [\dot{x}, \dot{y}, \dot{z}]^T$ is the linear velocity of the tool tip.
- $\omega = [\omega_x, \omega_y, \omega_z]^T$ is the angular velocity of the tool frame.

Because the physical posture of the robot changes as it moves, the mathematical relationship between the joints and the end-effector changes too. Therefore, the Jacobian is recalculated at every time step based on the current joint angles q .

Deriving the Jacobian from MDH Parameters

Because an explicitly expanded 6×6 symbolic Jacobian for a 6-DOF arm results in a massive trigonometric equation spanning several pages, we define it structurally using the **Vector Cross-Product Method** (the Geometric Jacobian).

To derive it, we first construct the transformation matrices T_i^{i-1} using the MDH parameters. For Modified Denavit-Hartenberg (Craig's convention), the transform from frame $i-1$ to frame i is:

$$T_i^{i-1} = \text{Rot}_X(\alpha_{i-1}) \text{Trans}_X(a_{i-1}) \text{Rot}_Z(\theta_i) \text{Trans}_Z(d_i)$$

Multiplying these step-by-step from the base gives the global transform for each frame:

$$T_i^0 = T_1^0 T_2^1 \dots T_i^{i-1} = \begin{bmatrix} R_i^0 & p_i^0 \\ 0 & 1 \end{bmatrix}$$

From each global transform T_i^0 we extract two pieces of geometric data:

1. p_i^0 : the 3D position of the origin of frame i .
2. z_i^0 : the axis of rotation for joint i (the third column of R_i^0).

For a revolute joint, its rotation causes the end-effector to sweep in an arc. The linear velocity generated is the cross product of the rotation axis and the lever arm connecting the joint to the end-effector; the angular velocity generated is simply the rotation axis itself. Therefore, the i -th column of the Jacobian is:

$$J_i = \begin{bmatrix} z_i^0 \times (p_{EE}^0 - p_i^0) \\ z_i^0 \end{bmatrix}$$

The Jacobian Matrix

Using the specific MDH parameters, the complete 6×6 base Jacobian is constructed column by column:

$$J(q) = [J_1 \quad J_2 \quad J_3 \quad J_4 \quad J_5 \quad J_6]$$

Expanded into linear (v) and angular (ω) components:

$$J(q) = \begin{bmatrix} z_1^0 \times (p_{EE}^0 - p_1^0) & z_2^0 \times (p_{EE}^0 - p_2^0) & z_3^0 \times (p_{EE}^0 - p_3^0) & z_4^0 \times (p_{EE}^0 - p_4^0) & z_5^0 \times (p_{EE}^0 - p_5^0) & z_6^0 \times (p_{EE}^0 - p_6^0) \\ z_1^0 & z_2^0 & z_3^0 & z_4^0 & z_5^0 & z_6^0 \end{bmatrix}$$

Applying the specific geometry:

- **Spherical wrist and tool offset:** As established in Forward Kinematics, $p_4^0 = p_5^0 = p_6^0$ (spherical wrist). The end-effector position includes the fixed tool transform:

$$p_{EE}^0 = p_6^0 + R_6^0 \begin{bmatrix} 0 \\ 0.000532 \\ 0.1447 \end{bmatrix}$$

When programming this for a real controller, the current z_i^0 and p_i^0 are computed from the MDH table at the current joint angles, substituted into the cross-product formula above, and evaluated to produce the 6×6 numerical matrix at each control step.

Kinematic Singularities

A singularity occurs when the Jacobian matrix loses rank (i.e. $\det(J) = 0$). In this state, the inverse Jacobian J^{-1} becomes undefined.

Physically, the robot loses one or more degrees of freedom: the end-effector becomes incapable of moving in certain Cartesian directions, no matter how fast the joint motors spin. Near a singularity, small required Cartesian movements command very large joint velocities, which can trigger safety faults in the controller.

Based on the MDH parameters (a 6-DOF arm with a spherical wrist), this robot exhibits three classic singularities:

1. **Wrist Singularity** ($q_5 = 0^\circ$ or 180°): Because joints 4, 5, and 6 form a spherical wrist, if joint 5 is straight, the axes of joint 4 (Z_4) and joint 6 (Z_6) become collinear. The robot loses the ability to rotate the end-effector about the axis perpendicular to Z_4/Z_6 , only the combined effect $q_4 \pm q_6$ remains controllable.
2. **Elbow Singularity** ($q_3 \approx 0^\circ$ or 180°): Occurs when the wrist center sits at the boundary of the arm's reachable radial distance from the shoulder, the arm fully outstretched ($L = a_3 + d_4$) or fully folded back on itself ($L = |a_3 - d_4|$). At this boundary, the 2-link shoulder-elbow sub-chain can still sweep tangentially but has lost all ability to move the wrist center radially inward or outward.
3. **Shoulder Singularity**: Occurs when the wrist center lies directly on the axis of joint 1 (Z_1). Rotating joint 1 does not change the position of the wrist center, effectively costing the robot a degree of freedom in translation.

Note: the robot's home configuration ($q = 0$) triggers *both* the wrist singularity ($q_5 = 0$) and the elbow singularity ($q_3 = 0$) simultaneously, since both angles happen to be zero at home. This was confirmed numerically (manipulability index = 0 at home). This is a common and deliberate characteristic of many industrial arm home poses, and is why teach pendants typically require the robot to be jogged off home before Cartesian-space jogging is enabled.

Workspace Trajectory

Five Cartesian waypoints were given (translation + quaternion, $[x, y, z, w]$), all sharing the same fixed tool orientation $[0, 1, 0, 0]$. Inverse kinematics was used to convert each waypoint into a joint configuration, and quintic-polynomial interpolation was used to generate a smooth, densely-sampled joint-space trajectory passing through all five.

Method

1. **Per-waypoint IK.** Each waypoint's target pose was solved using the closed-form IK derived in Analytical Inverse Kinematics, which returns up to 4 candidate solutions (elbow up/down $\times$ wrist flip/no-flip). At each waypoint, the branch closest to the previous waypoint's chosen solution was selected, so the arm does not jump between elbow or wrist configurations between waypoints.
2. **Interpolation.** Between each consecutive pair of waypoint joint configurations, a quintic polynomial was used to interpolate joint motion, with zero velocity and acceleration enforced at every waypoint. Four segments (5 waypoints) were generated, each sampled at 50 points, giving 197 total trajectory samples over an 8 s total duration.

Verification

Two checks were run on the resulting trajectory:

- **Waypoint accuracy:** forward kinematics evaluated at each of the 5 waypoint samples reproduces the original target pose to within 10^{-6} (position and orientation), confirming the IK solutions and interpolation endpoints are correct.
- **Velocity feasibility:** peak joint velocities across the full trajectory were checked against the URDF's per-joint velocity limits. All joints stay well within limits (peak: joint 1 at 26% of its limit; all others lower), confirming the trajectory is dynamically feasible at the chosen segment duration.

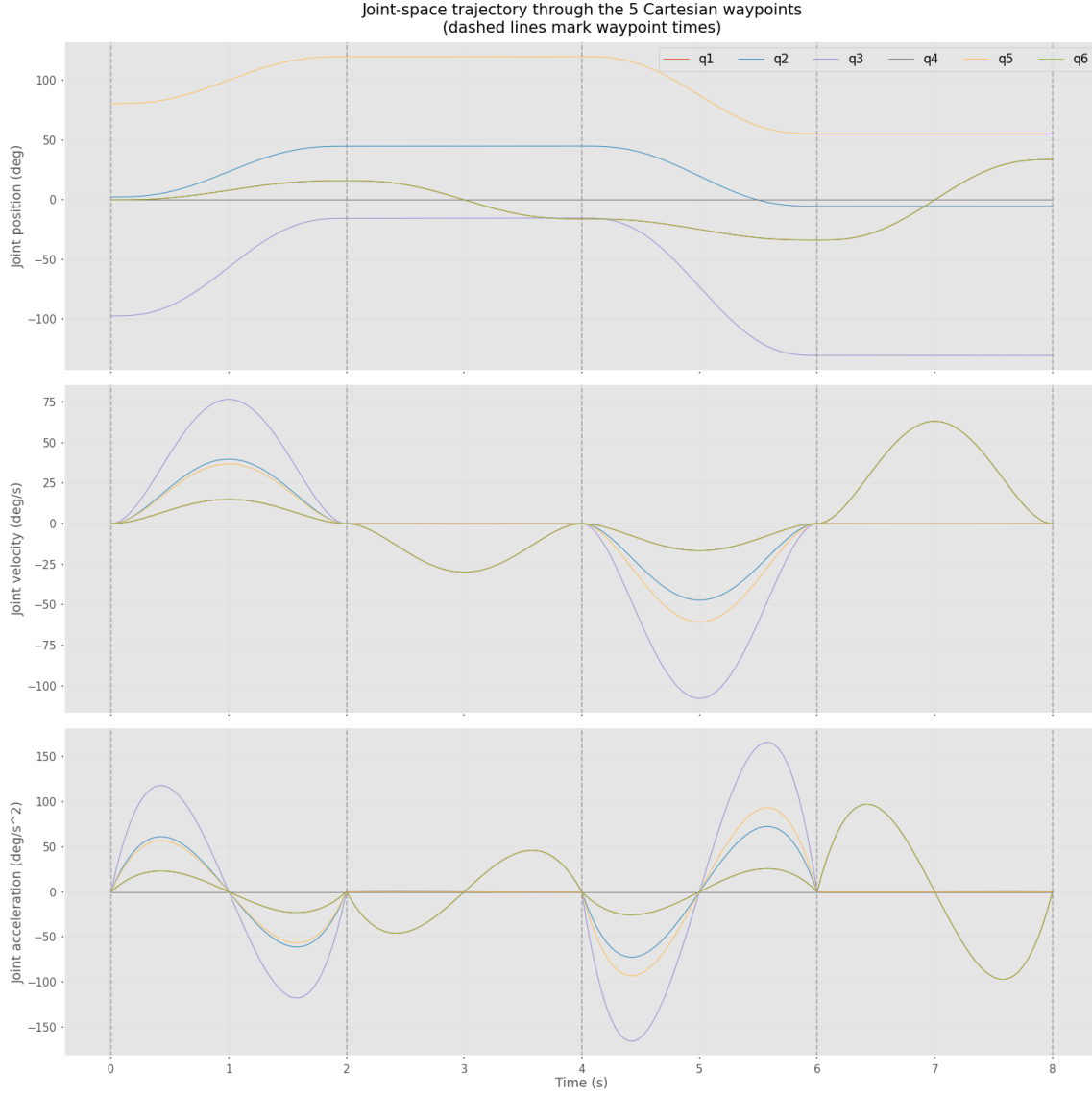


Figure 5: Joint-space trajectory (position, velocity, acceleration) through the 5 Cartesian waypoints. Dashed vertical lines mark the sample times corresponding to each of the 5 original waypoints.

Assumptions

- **Equal segment duration:** each inter-waypoint segment was allotted the same duration (2 s), rather than scaling segment time by Cartesian distance between waypoints. A more advanced version could scale duration by distance so the tool moves at a roughly constant Cartesian speed; this was not required for feasibility here, since all segments passed the velocity check comfortably.
- **Not time-optimal by design:** zero velocity/acceleration at every waypoint (including intermediate ones) means the arm briefly comes to rest at each one. This guarantees smoothness and avoids overshoot, and keeps each segment independently solvable, at the cost of overall trajectory time.

Visualization

For animation please refer videos/trajectories

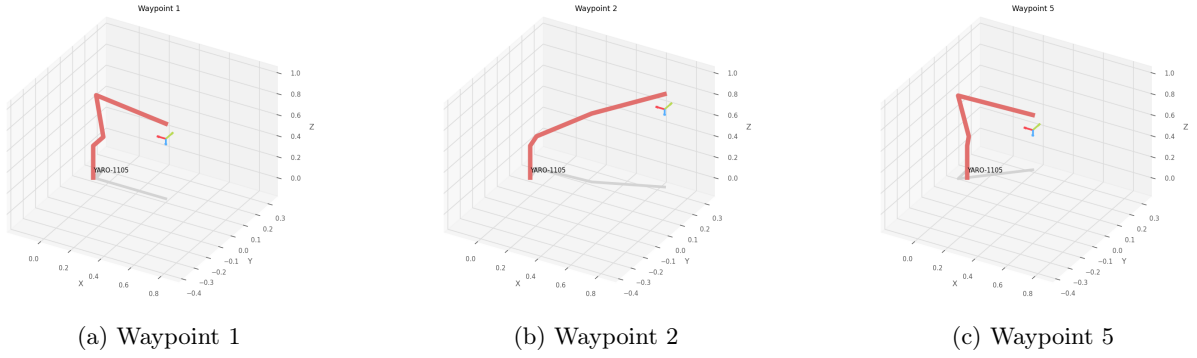


Figure 6: Robot configuration at representative waypoints along the trajectory.

Joint Torque Computation

Joint torque profiles were computed for the trajectory generated in Workspace Trajectory, using a custom Recursive Newton-Euler (RNEA) implementation built directly on the URDF's own per-link mass, center-of-mass, and inertia data (see Appendix for the full derivation, engine-choice justification, and verification). Two cases were computed: the robot following the trajectory unloaded, and the same trajectory carrying a 5 kg payload rigidly attached at the tool point.

Joint Torque Profiles: No Payload vs 5 kg Payload

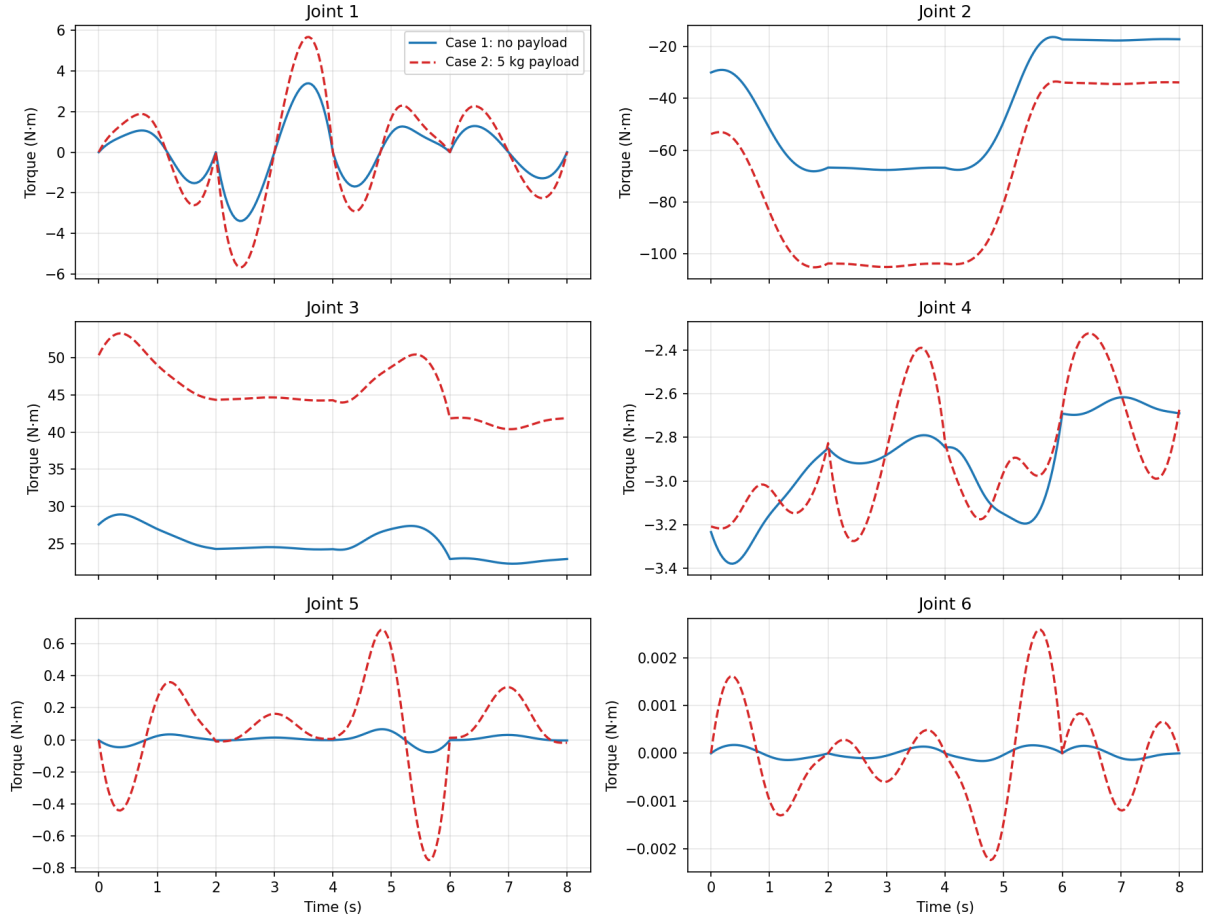


Figure 7: Joint torque profiles across the workspace trajectory: no payload (solid blue) vs. 5 kg payload at the tool point (dashed red).

Peak Torques

Joint	No payload (N·m)	5 kg payload (N·m)	Increase (N·m)
1	3.39	5.67	+2.28
2	68.10	105.13	+37.03
3	28.94	53.27	+24.33
4	3.38	3.27	-0.10
5	0.078	0.75	+0.67
6	0.0002	0.003	+0.002

Discussion

- **Joints 2–3 (shoulder, elbow) dominate**, via long lever arms ($a_2 = 0.49$ m, $d_4 = 0.4703$ m) carrying the payload's weight against gravity, gravity-driven offset with shape preserved.
- **Joint 1** increases moderately from inertial/Coriolis coupling only (no gravity torque about a vertical axis).
- **Joint 4 decreases slightly at its peak** — a reminder that added mass can shift *when* a peak occurs rather than uniformly raising it.
- **Joint 6 is essentially unaffected** (+0.002 N·m): the tool point sits nearly on joint 6's own axis, so payload has negligible lever arm there.

Takeaway: payload sensitivity concentrates almost entirely at the shoulder and elbow, not the wrist, relevant directly to motor sizing.

Appendix: RNEA Implementation Details

Dynamics Engine Choice

This section deliberately does *not* use `roboticstoolbox`'s built-in `robot.rne()` on the MDH-parameterized robot model. That path gives **incorrect** torques for this robot: 5 of the 6 MDH link frames are rotated relative to their corresponding URDF joint frames (only link 6's happens to coincide). Attaching a center-of-mass vector or inertia tensor taken directly from the URDF onto the corresponding MDH link – without first rotating it into that MDH frame's own axes – silently misdirects the link's gravity moment.

As a concrete check, at the home pose ($q = 0$, arm pointing straight up, so joint 2 should see negligible gravity moment):

<code>robot.rne()</code> on the MDH model:	$\tau_2 = 7.09 \text{ N}\cdot\text{m}$	(wrong)
Hand-derived RNEA (URDF-native frames):	$\tau_2 = 0.0065 \text{ N}\cdot\text{m}$	(correct)

Note this does *not* affect geometric derivations above: forward kinematics, inverse kinematics, and the Jacobian depend only on the geometric α, a, d, θ parameters, and FK computed through the MDH chain and the URDF-native chain agree bit-for-bit for the same q . It is specifically the *inertial* parameters that are frame-sensitive, which only matters once dynamics are introduced.

To avoid this pitfall entirely, joint torques are computed with a from-scratch Recursive Newton-Euler (RNEA) implementation built directly on the URDF's own per-link fixed transforms and joint-local z -axis rotations – sidestepping the MDH-frame remapping step, so there is nothing to get wrong there. This implementation was independently verified two ways:

1. **Static check:** at $\dot{q} = \ddot{q} = 0$, RNEA's gravity torque matches $\partial(\text{potential energy})/\partial q$ from a completely separate, energy-based computation, to $\sim 10^{-8}$.
2. **Dynamic check:** along an arbitrary smooth motion, instantaneous power $\tau \cdot \dot{q}$ matches the numerically-differentiated $d(KE + PE)/dt$, to $\sim 10^{-9}$ relative error – this exercises the inertia and Coriolis/centrifugal terms too, not just gravity.

The 5 kg payload is modeled as an extra zero-inertia point mass rigidly attached via a fixed (non-actuated) 7th frame at the tool point. RNEA naturally propagates its inertial effect back through the real joints – equivalent to merging it into link 6 via the parallel-axis theorem, without the manual matrix algebra that would require.

RNEA Algorithm (Pseudocode)

INITIALIZATION

```
omega_0      = [0, 0, 0]
omega_dot_0   = [0, 0, 0]
a_0          = [0, 0, 9.81]    // gravity trick: base "accelerates" upward
f_(n+1), n_(n+1) = forces/moments from payload at the tool point
                        (zero if no payload)
```

FORWARD PASS (i = 1 to N)

```
R = rotation matrix, link i-1 -> i
P = translation vector, link i-1 -> i (expressed in frame i-1)

omega_i      = R^T * omega_{i-1} + z_i * qdot_i
omega_dot_i   = R^T * omega_dot_{i-1}
                + (R^T * omega_{i-1}) x (z_i * qdot_i)
                + z_i * qddot_i

// NOTE: uses the PRE-joint-rate quantities (omega_{i-1}, omega_dot_{i-1}),
// rotated by R, with the ENTIRE bracket then rotated together --
// NOT the already-updated omega_i / omega_dot_i.
a_i   = R^T * [ a_{i-1} + (omega_dot_{i-1} x P) + (omega_{i-1} x (omega_{i-1} x P)) ]
a_c_i = a_i + (omega_dot_i x r_c) + (omega_i x (omega_i x r_c))
```

BACKWARD PASS (i = N down to 1)

```
F_i = m_i * a_c_i
N_i = I_i * omega_dot_i + omega_i x (I_i * omega_i)

R_next = rotation matrix, link i -> i+1
f_i = (R_next * f_{i+1}) + F_i
n_i = (R_next * n_{i+1}) + N_i + (r_c x F_i) + (P x (R_next * f_{i+1}))

tau_i = dot(n_i, z_i)    // only the axial component is the motor torque;
                        // the rest is reacted by the joint's bearings
```

RETURN tau // torques for all N actuated joints

Friction and rotor inertia terms are not included. This implementation and its verification cover only rigid-body dynamics.